

CRUD Operations with Spring Data JPA

```
HTTP Request
  ↓
Controller
  ↓
Service
  ↓
Spring Data JPA Repository
  ↓
JPA / Hibernate
  ↓
SQL
  ↓
Database
```

1. JPA vs Hibernate vs Spring Data JPA

Keep these three separate:

```
JPA
→ Specification/API for persistence

Hibernate
→ JPA implementation / ORM provider

Spring Data JPA
→ Repository abstraction on top of JPA
```

In a Spring Boot 3+ application, entities generally use:

```
import jakarta.persistence.*;
```

2. Entity

Example:

```
@Entity
@Table(name = "payments")
public class Payment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private BigDecimal amount;

    @Column(nullable = false)
```

```
private String status;

// constructors/getters/setters
}
```

Database:

```
payments
-----
id
amount
status
```

3. Repository

The usual starting point:

```
public interface PaymentRepository
    extends JpaRepository<Payment, Long> {
}
```

Now you automatically get methods such as:

```
save()
findById()
findAll()
existsById()
count()
deleteById()
delete()
```

You don't implement these methods yourself.

4. CREATE — `save()`

```
Payment payment = new Payment();

payment.setAmount(new BigDecimal("1500"));
payment.setStatus("PENDING");

Payment saved = paymentRepository.save(payment);
```

Conceptually Hibernate eventually executes:

```
INSERT INTO payments(amount, status)
VALUES (?, ?);
```

Then:

```
saved.getId();
```

can contain the generated database ID.

5. What Does `save()` Actually Do?

This is an important interview question.

Conceptually:

```
save(entity)
  ↓
Is entity new?
├─ YES → persist()
└─ NO  → merge()
```

Spring Data JPA decides whether the entity is new based on its entity-state detection strategy.

The underlying JPA operations are essentially:

```
entityManager.persist(entity);
```

or:

```
entityManager.merge(entity);
```

Important distinction

`persist()` :

Makes a new entity managed.

`merge()` :

Copies state from a detached/entity instance into a managed instance and returns the managed instance.

6. READ — `findById()`

```
Optional<Payment> payment =
    paymentRepository.findById(100L);
```

Typical usage:

```
Payment payment = paymentRepository
    .findById(id)
    .orElseThrow(() ->
        new PaymentNotFoundException("Payment not found"));
```

Conceptually:

```
SELECT *
FROM payments
```

```
WHERE id = ?;
```

7. Why Optional ?

It explicitly represents:

```
Record exists
    OR
Record does not exist
```

Instead of returning a potentially ambiguous `null`.

For an API:

```
findById()
  ↓
empty
  ↓
NotFoundException
  ↓
404 Not Found
```

8. READ — findAll()

```
List<Payment> payments =
    paymentRepository.findAll();
```

Conceptually:

```
SELECT *
FROM payments;
```

But never blindly use `findAll()` for a huge table.

If there are millions of records, use pagination.

9. Pagination

Repository:

```
Page<Payment> findAll(Pageable pageable);
```

Service:

```
Pageable pageable =
    PageRequest.of(0, 20);

Page<Payment> result =
    paymentRepository.findAll(pageable);
```

Conceptually:

```
page = 0  
size = 20
```

You can expose something like:

```
GET /payments?page=0&size=20
```

10. Page vs Slice

Page<T>

Provides information such as:

```
content  
totalElements  
totalPages  
current page
```

Slice<T>

Primarily answers:

```
Current data  
+  
Is there another slice?
```

Page may require a count query to determine total elements/pages.

So:

```
Need "Page 7 of 100" → Page  
Need "Load more"    → Slice
```

11. UPDATE — The Important Hibernate Way

Suppose:

```
@Transactional  
public Payment updateStatus(  
    Long id,  
    String status) {  
  
    Payment payment = repository  
        .findById(id)  
        .orElseThrow();  
  
    payment.setStatus(status);  
  
    return payment;  
}
```

Notice:

```
repository.save(payment);
```

isn't required just to make the change persistent when `payment` is already a **managed entity** inside the transaction.

Hibernate uses **dirty checking**.

Flow:

```
SELECT
↓
Managed Entity
↓
setStatus("PAID")
↓
Hibernate detects change
↓
flush
↓
UPDATE
↓
COMMIT
```

Generated SQL is conceptually:

```
UPDATE payments
SET status = ?
WHERE id = ?;
```

12. Does `save()` Also Work for Update?

Yes.

You may see:

```
payment.setStatus("PAID");
repository.save(payment);
```

That's valid.

But understand:

`save()` isn't what makes dirty checking work.

A managed entity is already tracked by the persistence context.

This distinction is often tested in Hibernate interviews.

13. DELETE — `deleteById()`

```
paymentRepository.deleteById(id);
```

Conceptually:

```
DELETE FROM payments
WHERE id = ?;
```

You may explicitly validate existence first:

```
if (!repository.existsById(id)) {
    throw new PaymentNotFoundException("Not found");
}

repository.deleteById(id);
```

Whether this extra query is desirable depends on your API semantics and database behavior.

14. `existsById()`

If you only need:

Does the record exist?

use:

```
boolean exists =
    repository.existsById(id);
```

instead of loading the entire entity.

15. `count()`

```
long count = repository.count();
```

Conceptually:

```
SELECT COUNT(*)
FROM payments;
```

16. CRUD Summary

Operation	Repository method	Typical HTTP
Create	<code>save()</code>	POST
Read one	<code>findById()</code>	GET

Read one	findById()	GET
Read all	findAll()	GET
Update	managed entity / save()	PUT/PATCH
Delete	deleteById()	DELETE
Exists	existsById()	—
Count	count()	—

17. Derived Query Methods

One of Spring Data JPA's biggest advantages is query derivation.

```
List<Payment> findByStatus(String status);
```

Spring generates the query.

Conceptually:

```
SELECT *
FROM payments
WHERE status = ?;
```

More complex:

```
List<Payment> findByStatusAndAmountGreaterThan(
    String status,
    BigDecimal amount);
```

Equivalent conceptually to:

```
WHERE status = ?
AND amount > ?
```

18. Common Derived Query Keywords

Know these:

```
findBy
findAllBy
countBy
existsBy
deleteBy

And
Or

GreaterThan
LessThan
Between

Like
```

```
Containing
StartingWith
EndingWith

OrderBy
```

Example:

```
List<Payment> findByStatusOrderByAmountDesc(
    String status);
```

19. When Derived Queries Become Ugly

Don't create giant method names such as:

```
findByStatusAndVendorAndAmountGreaterThanAndCreatedAtBetweenAndCustomerId(...)
```

Use `@Query` when the query becomes complex.

```
@Query("""
    SELECT p
    FROM Payment p
    WHERE p.status = :status
    AND p.amount > :amount
""")
List<Payment> findEligiblePayments(
    @Param("status") String status,
    @Param("amount") BigDecimal amount);
```

That's **JPQL**, not SQL.

20. JPQL vs SQL

JPQL works primarily with:

```
Entities
Fields
Relationships
```

Example:

```
SELECT p
FROM Payment p
WHERE p.status = :status
```

Here:

```
Payment → Entity
p.status → Java entity field
```

SQL uses tables/columns:

```
SELECT *
FROM payments
WHERE status = ?;
```

Hibernate converts the JPA/JPQL operation into SQL appropriate for the configured database.

21. Native Query

You can explicitly use SQL:

```
@Query(
    value = """
        SELECT *
        FROM payments
        WHERE status = :status
    """,
    nativeQuery = true
)
List<Payment> findPayments(
    @Param("status") String status);
```

Use native SQL when there is a concrete reason, such as:

- database-specific functionality
- complex SQL
- specialized performance requirements
- features not conveniently expressed in JPQL

Don't make every query native by default.

22. Custom UPDATE / DELETE Queries

For bulk modifications:

```
@Modifying
@Query("""
    UPDATE Payment p
    SET p.status = :status
    WHERE p.vendor = :vendor
""")
int updateStatus(
    @Param("status") String status,
    @Param("vendor") String vendor);
```

Typically used inside a transaction:

```
@Transactional
```

`@Modifying` tells Spring Data this query modifies data rather than being a normal select.

23. Important: Bulk Update vs Normal Update

Normal entity update

```
Payment payment = repository.findById(id)
    .orElseThrow();

payment.setStatus("PAID");
```

Hibernate knows the entity is managed.

Bulk query

```
UPDATE Payment p
SET p.status = 'PAID'
WHERE ...
```

This operates directly at the database level and can make already-managed entities in the persistence context stale.

That's why bulk updates need more care.

24. `save()` vs `saveAndFlush()`

```
repository.save(payment);
```

doesn't necessarily mean:

SQL has already been sent to the DB and committed.

Think:

```
save
↓
Persistence Context
↓
flush
↓
SQL sent
↓
transaction commit
```

You can explicitly flush:

```
repository.flush();
```

or:

```
repository.saveAndFlush(payment);
```

Critical distinction

flush ≠ commit

A flush sends synchronization SQL to the database, but the surrounding transaction can still roll back.

25. Service Layer + Repository

A clean backend structure is:

```
@Service
public class PaymentService {

    private final PaymentRepository repository;

    public PaymentService(PaymentRepository repository) {
        this.repository = repository;
    }

    @Transactional
    public Payment create(Payment payment) {
        return repository.save(payment);
    }

    @Transactional(readOnly = true)
    public Payment get(Long id) {
        return repository.findById(id)
            .orElseThrow(() ->
                new PaymentNotFoundException(
                    "Payment not found: " + id));
    }

    @Transactional
    public Payment updateStatus(
        Long id,
        String status) {

        Payment payment = repository.findById(id)
            .orElseThrow();

        payment.setStatus(status);

        return payment;
    }

    @Transactional
    public void delete(Long id) {
        if (!repository.existsById(id)) {
            throw new PaymentNotFoundException(
                "Payment not found");
        }

        repository.deleteById(id);
    }
}
```

26. Controller

For production APIs, use DTOs rather than exposing entities directly.

```

@RestController
@RequestMapping("/payments")
public class PaymentController {

    private final PaymentService service;

    public PaymentController(PaymentService service) {
        this.service = service;
    }

    @PostMapping
    public PaymentResponse create(
        @RequestBody PaymentRequest request) {

        return service.create(request);
    }

    @GetMapping("/{id}")
    public PaymentResponse get(
        @PathVariable Long id) {

        return service.get(id);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(
        @PathVariable Long id) {

        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}

```

The architectural flow should be:

```

Controller
↓
DTO
↓
Service
↓
Entity
↓
Repository
↓
Hibernate
↓
Database

```

27. Why Not Expose Entity Directly?

Returning:

```
Payment
```

directly from your REST controller can cause:

- persistence model leaking into API
- unwanted fields exposed
- lazy-loading issues

- bidirectional JSON recursion
- unstable API contracts

Prefer:

```
PaymentResponse
```

for the API layer.

28. JpaRepository vs CrudRepository

CrudRepository

Basic CRUD functionality.

JpaRepository

Provides JPA-oriented capabilities in addition to CRUD, including:

```
flush()
saveAndFlush()
batch-related operations
pagination/sorting support through its hierarchy
```

Typical Spring Boot application:

```
public interface PaymentRepository
    extends JpaRepository<Payment, Long> {
}
```

29. findById() vs getReferenceById()

findById()

```
Optional<Payment> payment =
    repository.findById(id);
```

Good when you actually need to read/validate the entity.

getReferenceById()

```
Payment payment =
    repository.getReferenceById(id);
```

Can give you a reference/proxy without immediately loading the complete entity.

A practical use:

```
payment.setCustomer(
    customerRepository.getReferenceById(customerId)
);
```

when you only need to establish a relationship.

But don't use it blindly to avoid queries; if you access data on that reference, Hibernate may still need to hit the database.

30. N+1 and CRUD

Suppose:

```
List<Customer> customers =
    customerRepository.findAll();

for (Customer c : customers) {
    c.getPayments().size();
}
```

You may get:

```
1 query → customers
N queries → payments for each customer
```

```
1 + N = N+1 queries
```

This is a very common Hibernate performance problem.

Possible solutions include:

```
JOIN FETCH
@EntityGraph
batch fetching
proper query design
```

Don't solve everything by changing associations to `EAGER`.

31. Transaction + CRUD

Suppose:

```
@Transactional
public void process() {

    paymentRepository.save(payment);

    invoiceRepository.save(invoice);

    historyRepository.save(history);
}
```

The intended database transaction is:

```
BEGIN
↓
INSERT payment
```

```
↓  
UPDATE invoice  
↓  
INSERT history  
↓  
COMMIT
```

If an appropriate failure causes rollback:

```
ROLLBACK
```

all relevant database changes in that transaction are rolled back.

This is why transaction boundaries are generally placed around the **business operation**, often in the service layer rather than around individual repository calls.

32. Real Interview Scenario

Interviewer:

"What happens when I call `PUT /payments/101` to change a payment's status?"

A strong answer:

```
HTTP PUT  
↓  
Controller  
↓  
Service  
↓  
@Transactional  
↓  
repository.findById(101)  
↓  
Hibernate executes SELECT  
↓  
Payment becomes managed  
↓  
payment.setStatus("PAID")  
↓  
Dirty checking  
↓  
flush  
↓  
UPDATE payments ...  
↓  
transaction commit  
↓  
HTTP response
```

This is much stronger than:

"We call `save()` and update the database."

33. Most Important Interview Questions

Repository

- What is Spring Data JPA?
- Why extend `JpaRepository` ?
- `JpaRepository` vs `CrudRepository` ?
- What methods are inherited automatically?

`save()`

- How does `save()` decide INSERT vs UPDATE?
- `persist()` vs `merge()` ?
- Does `save()` immediately execute SQL?
- `save()` vs `saveAndFlush()` ?

Queries

- What are derived query methods?
- How does `findByStatusAndAmountGreaterThan()` work?
- JPQL vs SQL?
- `@Query` vs native query?
- What is `@Modifying` ?

Hibernate

- What is the persistence context?
- What is dirty checking?
- Does updating a managed entity require `save()` ?
- What is N+1?
- How do you solve N+1?

Transactions

- Where should `@Transactional` be used?
- What does `readOnly = true` mean?
- Flush vs commit?
- What happens if a repository call fails halfway through a service operation?

34. BPS-Specific Questions — Inspect Your Codebase

Now connect this to your actual BPS implementation.

Search your codebase for:

```
extends JpaRepository
@Transactional
save{
findBy
@Query
@Modifying
```

```
saveAndFlush  
flush  
findAll(  
Pageable
```

Then answer these from the actual code:

Repository

1. Which BPS entities extend `JpaRepository` ?
2. What are their ID types?
3. Which repository methods are inherited?
4. Which custom query methods exist?

CRUD flow

Trace one real operation:

```
BPS API  
↓  
Controller  
↓  
Service  
↓  
Repository  
↓  
Hibernate  
↓  
SQL  
↓  
Database
```

For example, inspect how a payment record is:

```
created  
retrieved  
updated  
deleted
```

Critical interview question

When BPS updates a payment entity, does the code explicitly call `repository.save()` after changing the entity, or does it rely on Hibernate dirty checking inside `@Transactional` ? Why?

That answer will show whether you actually understand the persistence layer rather than just knowing Spring Data method names.